Laboratorio 09

Bag of Words, Similitud de Coseno e Índice Invertido

Juan Marcos Guerrero Cueva Adrian Urbina Mendoza
Hector Huaman Puiquin

Base de Datos II – Prof. Heider Sanchez – 2026-2

16 de octubre de 2026

Índice

1. P1. Configuración y carga de datos 1
2. P2. Preprocesamiento y Bag of Words 1
3. P3. Búsqueda booleana 2
4. P4. Similitud de coseno 2
5. P5. Experimento 2
6. P6. Análisis de resultados 4

1. P1. Configuración y carga de datos

Se usó PostgreSQL 17.5 y Python 3.13 (pandas, psycopg2, NLTK, Matplotlib). El notebook `lab09.ipynb` crea la base y las tablas `noticias` y `stopwords` con el esquema del enunciado, y carga `news_es.csv` (1217 noticias) y `stoplist.txt` (608 stopwords). Solo inserta datos si las tablas están vacías, así que se puede volver a ejecutar sin duplicar filas. Las credenciales se leen de un archivo `.env`, que no se sube al repositorio.

2. P2. Preprocesamiento y Bag of Words

`preprocess` aplica cuatro pasos: minúsculas, tokenización (solo secuencias de letras, sin números ni signos), eliminación de las stopwords leídas de la tabla y stemming Snowball para español. `compute_bow` cuenta la frecuencia de cada término y `update_bow_in_db` guarda el resultado como JSONB con un `UPDATE` parametrizado. Las consultas pasan por la misma función, de modo que *economía* y *económico* terminan en el mismo término `econom`.

```
def preprocess(text):
    if not isinstance(text, str): # NULL o vacío
        return []
    text = text.lower() # 1. minúsculas
    tokens = re.findall(r"[a-záéíóúüñ]+", text) # 2. tokenización
    tokens = [t for t in tokens if t not in STOPWORDS] # 3. stopwords
    return [stemmer.stem(t) for t in tokens] # 4. stemming
```

3. P3. Búsqueda booleana

Cada consulta se convierte en un árbol, por ejemplo:

`tecnología AND (banca OR finanzas)` → `('AND', 'tecnolog', ('OR', 'banc', 'finanz'))`

Los operadores se evalúan de izquierda a derecha y los paréntesis agrupan. La versión **lineal** evalúa el árbol sobre el BoW de cada documento. La versión con **índice** lo evalúa sobre las listas `(doc_id, tf)` con `AND` (intersección), `OR` (unión) y `AND_NOT` (diferencia). El notebook también ejecuta las pruebas con la forma `idx.showDocuments(AND(idx.L(...), ...))` del enunciado. La tabla 1 muestra las 9 consultas: las 6 del enunciado y 3 nuevas, marcadas con *.

Consulta	Resultados	Lineal = Índice
economía AND inflación	244	Sí
banca OR finanzas	547	Sí
tecnología AND-NOT inflación	306	Sí
sostenibilidad AND ambiente AND renovables	9	Sí
tecnología AND (banca OR finanzas)	174	Sí
economía AND-NOT inflación	358	Sí
* (banca OR finanzas) AND-NOT inflación	368	Sí
* economía AND (tecnología OR sostenibilidad)	212	Sí
* (renovables OR ambiente) AND-NOT economía	65	Sí

Cuadro 1: Pruebas booleanas sobre las 1217 noticias.

4. P4. Similitud de coseno

Peso TF-IDF: $w_{t,d} = \log_{10}(1 + \mathrm{tf}_{t,d}) \cdot \log_{10}(N/\mathrm{df}_t)$, y score $= \frac{\sum_t w_{t,q}\, w_{t,d}}{\|q\|\,\|d\|}$. La versión **lineal** recalcula df, idf y normas recorriendo todos los BoW en cada consulta, sin usar el índice, y compara la consulta con cada documento. La versión con **índice** usa idf y normas precalculados y solo acumula score para los documentos que aparecen en las listas de los términos de la consulta. Las dos versiones usan la misma función para calcular idf y normas, y los empates se ordenan por id.

Consulta	Top-5 (id)	Dif. máx. de score
innovaciones en banca digital y tecnología financiera	390, 1097, 829, 282, 220	0
inflación y crecimiento económico	96, 143, 1214, 783, 198	2.8×10^{-17}
sostenibilidad y energías renovables	661, 620, 150, 189, 713	2.8×10^{-17}

Cuadro 2: Top-5 por coseno. Lineal e índice devolvieron los mismos documentos en el mismo orden.

Además, una celda de pruebas con 4 documentos de juguete verifica a mano los casos borde: término inexistente, consulta vacía, `top_k` mayor que la cantidad de resultados y un término con idf = 0.

5. P5. Experimento

Se usaron los primeros N documentos por id, con $N \in \{10, 100, 250, 500, 750, 1000, 1217\}$. Para cada tamaño se construye un índice propio sobre el mismo subconjunto que recorre la versión lineal, y antes de medir se verifica que ambas den el mismo resultado. Cada consulta (9 booleanas y 3 de coseno) se ejecuta una vez de calentamiento y luego se mide **30 veces** con `time.perf_counter()`. Las curvas solo incluyen el tiempo de consulta; la construcción del índice se mide aparte (promedio de 5 construcciones). Los datos están en `resultados/tiempos.csv`.

N	Bool. lineal	Bool. índice	Cos. lineal	Cos. índice	Construcción
10	0.000117	0.000036	0.000734	0.000047	0.000865
100	0.000195	0.000044	0.006646	0.000096	0.010964
250	0.000311	0.000057	0.016612	0.000167	0.027359
500	0.000517	0.000075	0.032517	0.000310	0.072992
750	0.000737	0.000103	0.048272	0.000454	0.102433
1000	0.000961	0.000124	0.063357	0.000580	0.127602
1217	0.001209	0.000149	0.077862	0.000712	0.152287

Cuadro 3: Tiempo promedio por consulta y tiempo de construcción del índice, en segundos.

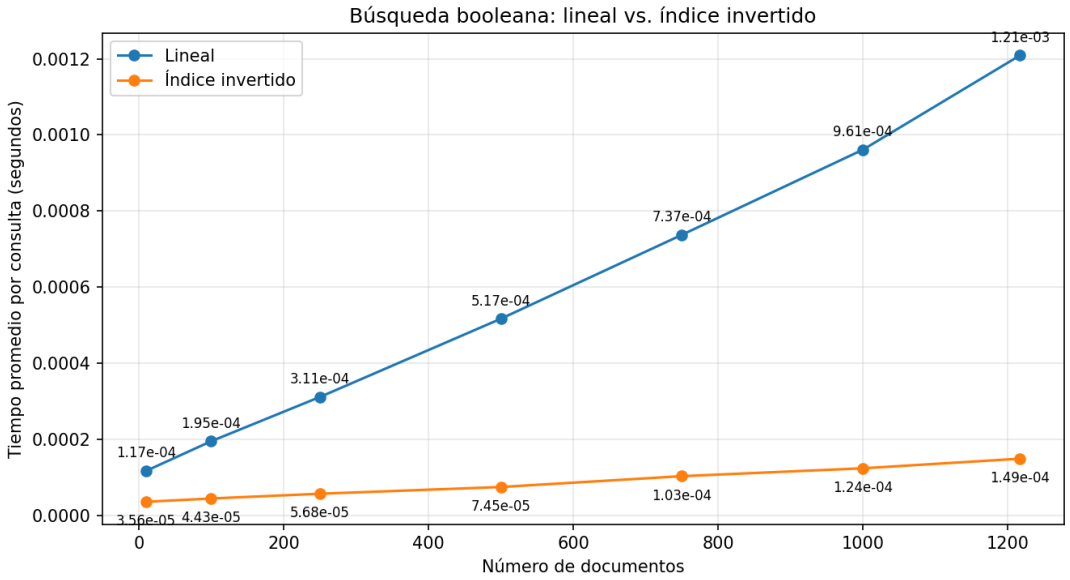


Figura 1: Búsqueda booleana: lineal vs. índice invertido.

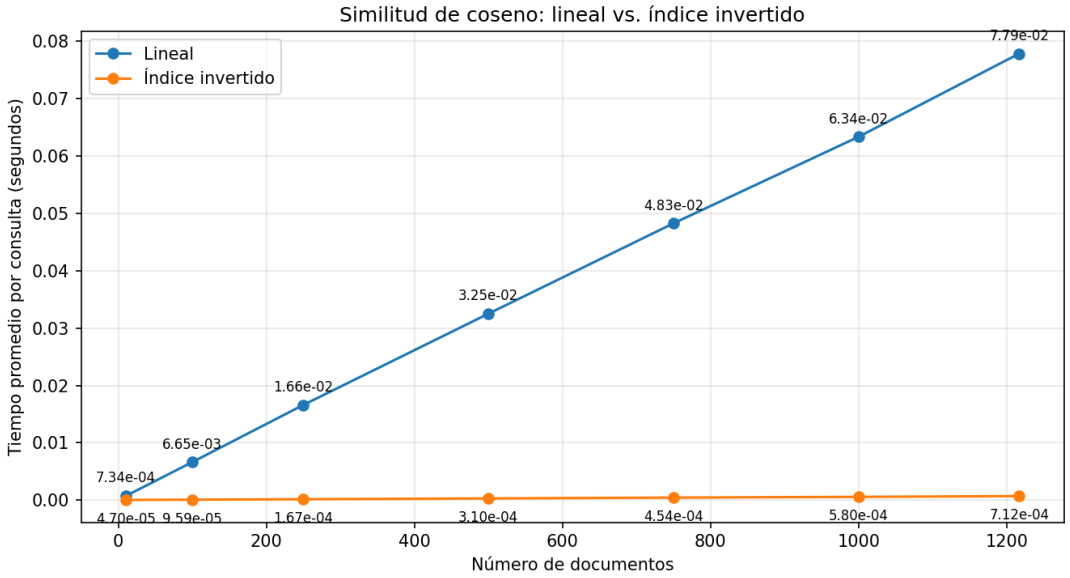


Figura 2: Similitud de coseno: lineal vs. índice invertido.

6. P6. Análisis de resultados

1. Complejidad. La booleana lineal es $O(N \cdot q)$, porque revisa el BoW de cada documento. La booleana con índice es $O(\sum_{t \in q} |L(t)|)$, porque solo recorre las listas de los términos consultados. El coseno lineal es $O(P)$, donde P es el total de pares (término, documento), ya que en cada consulta recalcula df, idf y normas. El coseno con índice es $O(\sum |L(t)| + C \log C)$, donde C es el número de candidatos. Los datos lo confirman. De $N = 10$ a $N = 1217$ (122 veces más documentos), el coseno lineal crece 106 veces (de 0.00073 s a 0.0779 s), casi proporcional a N. La booleana lineal crece 10 veces (de 0.00012 s a 0.00121 s), porque con N pequeño pesa el costo fijo de analizar la consulta. Con índice el crecimiento es mucho menor: 4 veces en booleana y 15 en coseno.

2. Punto de quiebre. En tiempo de consulta, el índice gana desde el tamaño más pequeño medido ($N = 10$) en ambos casos: 3.3 veces más rápido en booleana y 15.6 en coseno, así que los puntos coinciden. Si se suma la construcción, ya no coinciden. En coseno, el índice se paga con unas 2 consultas. En booleana hacen falta entre 11 consultas ($N = 10$) y 144 ($N = 1217$).

3. Overhead. Sí: la construcción del índice. Toma 0.87 ms con $N = 10$ y 152 ms con $N = 1217$, más que una búsqueda booleana lineal (0.12 ms y 1.21 ms). Ese tiempo se va en recorrer todos los BoW, armar las listas y calcular idf y normas. Con pocas consultas booleanas, el índice no conviene.

4. Exactitud. Las 9 consultas booleanas devolvieron los mismos IDs con ambos enfoques en los 7 tamaños. En coseno coinciden el top-5 y su orden. La diferencia máxima de score es 2.8×10^{-17}, porque las sumas en coma flotante se hacen en distinto orden: el índice suma por término y el lineal por documento. Los empates (por ejemplo, los documentos 96 y 143) se resuelven por id.

5. Limitaciones. Hoy el índice tiene 13 330 términos y 180 214 postings. Con 100 veces más noticias tendría unos 18 millones de postings en tuplas de Python, es decir, varios GB de RAM. Además, habría que reconstruirlo en cada ejecución, y eso pasaría de 0.15 s a decenas de segundos. La alternativa es un índice persistente en disco: una tabla de PostgreSQL `(termino, doc_id, tf)` con índice B+Tree sobre `termino`, o un índice GIN sobre `tsvector`. Así solo se leen las listas que la consulta necesita.